Receiver-Guided Reinforcement Learning-based Congestion Control for IoT

Qihang Jiao, *Student Member, IEEE,* and Lotfi Mhamdi, *Senior Member, IEEE*

***Abstract*—The abstract goes here.**

***Index Terms*—IoT, TCP, congestion control, reinforcement learning.**

I. Introduction

THIS part is to be completed.

II. Related Works

This part is to be completed.

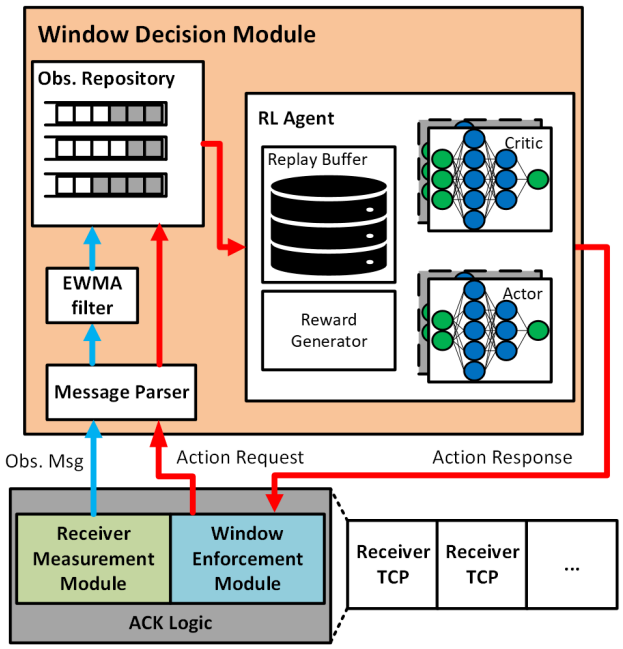


Fig. 1: System architecture of our proposal.

III. System Design

Our proposal is designed to be receiver-guided, and not blocking normal TCP behaviour. Therefore, we extended and modified the standard TCP to exchange message between receiver and sender, as well as between the receiver and agent. We introduce our control message design and the information exchange mechanism to enable receiver-guided congestion control for TCP. Then we present three modules in the receiver side for adjusting sender $cwnd$ relying on the receiver-guided method i.e., Receiver Measurement Module, Window Decision Module, and Window Enforcement Module.

Qihang Jiao and Lotfi Mhamdi are with the School of Electronic and Electrical Engineering, University of Leeds, Leeds, UK (e-mail: el22qj@leeds.ac.uk; L.Mhamdi@leeds.ac.uk).

Lotfi Mhamdi is also with the College of Computing & Systems, Abdullah Al Salem University, Kuwait (e-mail: lotfi.mhamdi@aasu.edu.kw).

A. Enable Receiver-Guided for TCP

1) Control Message: The receiver TCP socket is extended to support communication with the RL agent. In our design, generating an observation and an action are decoupled to ensure the packet communication will not be blocked by the agent inferrence. We define three types of control messages as following:

- **Observation Message**: carries the observed network status and is sent to the agent for storage in the replay buffer.
- **Action Request**: triggers the agent to infer and return a control action based on the current state.
- **Connection Close**: notifies the agent that a flow has terminated, allowing it to clear the corresponding internal states.

2) Exchange information through Modified TCP: We utilise the window size field in TCP header to exchange the information required both by the receiver and the sender. In particular, the sender TCP is modified to include the current $cwnd$ value in the window size field of TCP header. For an enforced $cwnd$, when the receiver TCP receives an control signal from the Window Decision Module, it sets its state as control start, and keeps including the enforced $cwnd$ value into the window size field of ACK with **ECE**. When the sender receives an **ECE** ACK, it enters a window adjustment phase and overwirtes its current $cwnd$ to the enforced value. During this phase, replicative ACKs with **ECE** are ignored by the sender. Once the window adjustment is completed, the sender TCP will keep sending **CWR** packet and exit the window adjustment phase. Then, the receiver will reply with a **CWR** and both sender and receiver will exit the control phase. By applying the mechanism above, our design can achieve a value exchange that will not be lost due to packet loss that makes it possible for applying to multiple flows.

B. Receiver Measurement Module

Our scheme relies on reliable state observations to ensure stable training. Therefore, the states in our design are required to reflect the congestion extent while remaining observable at the receiver. Despite RTT is a fine-grained and valuable indicator of congestion, it is difficult to directly observe in the receiver side.

To address the limitation above, we derive a receiver-side congestion extent metric based on the $cwnd$ elapsed time

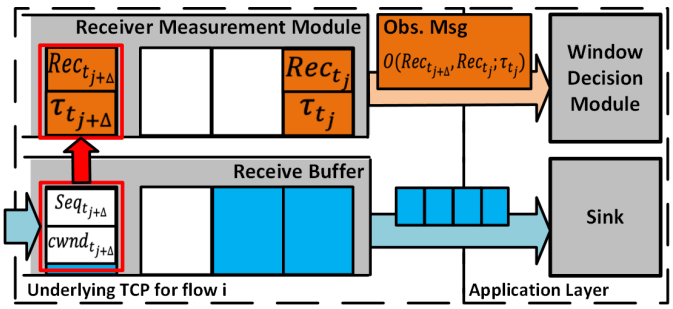


Fig. 2: An example of estimation task completion of flow i.

($t_{elapsed}$). We extend the sender TCP feature to include its $cwnd$ in TCP header. In standard TCP congestion avoidance, the congestion window $cwnd$ is increased by 1 MSS after an entire $cwnd$ bytes has been acknowledged. This nature results in an approximate growth rate of 1 MSS per RTT, in which the method is named Additive Increment.

We utilise Additive Increment growing feature to enable the receiver to estimate network congestion extent by calculating the elapsed time for a flow to complete its $cwnd$. In particular, upon every packet receive event that includes a $cwnd$ value different from the last value received, the Receiver Measurement Module establishes a new estimation task.

A new task is initialised with a record (Rec) for the current flow state and a threshold (τ), e.g., for flow i started at time t_0 and estimated at time t_j, defined as below:

$$Rec_{i,t_j} \triangleq [B_i(t_j), L_i(t_j), t_j] \tag{1}$$

where we set τ to be the sum of sequence number (Seq_{i,t_j}) and the $cwnd$ value from its header. We also maintain a cumulative received bytes ($RxBytes_{i,t}$) and cumulative packet losses ($Losses_{i,t}$) counter in the receiver TCP, defined as follows:

$$\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j} \tag{2}$$

$$B_i(t_j) = \sum_{n=t_0}^{t_j} RxBytes_{i,n} \tag{3}$$

$$L_i(t_j) = \sum_{n=t_0}^{t_j} Losses_{i,n} \tag{4}$$

An estimation task is completed at $t_{j+\Delta}$ when a coming packet is seen with a sequence number satisfying $Seq_{i,t_{j+\Delta}} > \tau_{i,t_j}$. We calculate this task duration as the $cwnd$ elapsed time i.e., $t_{elapsed} = t_{j+\Delta} - t_j$. We then calculate the goodput (g_{i,t_j}) and packet losses during the estimation task, where $g_{i,t_j} = \frac{B_i(t_{j+\Delta}) - B_i(t_j)}{t_{elapsed}}$. and $\ell_{i,t_j} = L_i(t_{j+\Delta}) - L_i(t_j)$. Therefore, an observation is formed denoted as $o_{i,t_j} = O(Rec_{i,t_{j+\Delta}}, Rec_{i,t_j}; \tau_{i,t_j})$, with the observation as the result of the generation process defined as follows:

$$O(Rec_{i,t_{j+\Delta}}, Rec_{i,t_j}; \tau_{i,t_j}) = \left[g_{i,t_j}, \ell_{i,t_j}, t_{elapsed}\right]$$

The eventual observation is encapsulated into a Observation Message and sent to the Window Decision Module for storage. The consecutive process triggered by packet received event can be seen in Algorithm 1.

Algorithm 1 Receiver Measurement and Observation Generation

Input: Flow Id i, Time t_j, Arrived Packet p_{i,t_j}, Flow Observation Queue q_i
Output: Observation o_{i,t_j}, Control Message M

Extract Seq_{i,t_j}, $cwnd_{i,t_j}$, and $RxBytes_{i,t_j}$ from p_{i,t_j}
$\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j}$
Obtain $Losses_{i,t_j}$ from $TcpSocket_i$
$B_i(t_j) = B_i(t_{j-1}) + RxBytes_{i,t_j}$
$L_i(t_j) = L_i(t_{j-1}) + Losses_{i,t_j}$
$Rec_{i,t_j} = [B_i(t_j), L_i(t_j), t_j]$
Obtain Rec_i^{head} and τ_i^{head} from the head of q_i
if $Seq_{i,t_j} \geq \tau_i^{head}$ **then**
    Dequeue Rec_i^{head} and τ_i^{head} from q_i
    $o_{i,t_j} = O(Rec_{i,t_j}, Rec_i^{head})$
    Include o_{i,t_j} into M
    Send M to agent
else if $cwnd_{i,t_j} \neq cwnd_{i,t_{j-1}}$ **then**
    Enqueue Rec_{i,t_j} and τ_{i,t_j} into q_i
end if

Algorithm 2 Window Decision Module

Input: Control Message M, Replay Buffer B, Flow State Buffer B_i for flow i, Batch Size N
Output:

if M = "Connnection Close" **then**
    Clear B_i
else if M = "State" **then**
    Extract s_i from M
    Store s_i into B_i
else if M = "Action Request" **then**
    Compute the average state $\bar{s_i}$ over B_i
    $a = \mu(\bar{s_i}; \theta^{\mu})$
    Store transition $(\bar{s_i}, a_i, r_i, \bar{s_i}')$ into B
    Randomly sample N transition from B
    Compute $y_t = r(s_t, a_t) + \gamma Q^-(s_{t+1}, \mu^-(s_{t+1}))$
    Compute $L(\theta^Q) = \mathbb{E}[(Q(s_t, a_t; \theta^Q) - y_t)^2]$
    Update Critic minimising $L(\theta^Q)$
    Compute $\nabla_{\theta^\mu} J(\theta^\mu) = \mathbb{E}[\nabla_{a_t} Q(s_t, \mu(s_t)) \nabla_{\theta^\mu} \mu(s_t)]$
    Update Actor via gradient ascent $\nabla_{\theta^\mu} J(\theta^\mu)$
    $\theta^{\mu^-} \leftarrow \tau \times \theta + (1 - \tau) \times \theta^{\mu^-}$
    $\theta^{Q^-} \leftarrow \tau \times \theta + (1 - \tau) \times \theta^{Q^-}$
end if

C. Window Decision Module

Window Decision Module consists of a Reinforcement Learning model to output a scaling factor to underlying TCP socket periodically triggered by Action Request from Window Enforcement Module. An enforced congestion window is then calculated with the factor by the underlying TCP socket and included in ACK to the sender. The sender will overwrite its $cwnd$ by the enforced congestion window. Therefore, to implement a continuous control to modified TCP, we choose Deep Deterministic Policy Gradient (DDPG) as agent in our design.

1) State Generation: An observation is generated on the packet arrival event while the action is requested periodically. In our design, the action period is set to long enough to ensure we have multiple observations stored by an Observation Repository.Therefore, we first use an exponential weighted moving average (EWMA) filter to smooth the observations before storing them in the repository. This is done to mitigate the noise when the observation is generated in a varying network environment e.g., Wi-Fi. After storing, when an action request is received, the average of the stored observations during this action period is calculated as input to our agent, seen as follows when n observations are stored:

$$s_t = \left[\frac{1}{n}\sum_{i=1}^{n} g_i, \frac{1}{n}\sum_{i=1}^{n} t_{elapsed,i}\right] \tag{5}$$

2) Reward Design: For improving network performance for each flow, the first component of our reward is based on network power metric to reflect individual flow performance, defined as follows:

$$r_{1,t} = power = \frac{\frac{1}{n}\sum_{i=1}^{n} g_i}{\frac{1}{n}\sum_{i=1}^{n} t_{elapsed,i}}$$

We also take the advantage of the global performance store in repository to designe the second component for improving fairness among flows. Jain's fairness index is introduced considring N flows and $\bar{g_i} = \frac{1}{n}\sum_{i=1}^{n} g_{i,t}$ defined as follows:

$$r_{2,t} = J(\bar{\mathbf{g}}) = \frac{\left(\sum_{i=1}^{N} \bar{g_i}\right)^2}{N\sum_{i=1}^{N} \bar{g}_i^2}$$

Eventually, we design our reward as:

$$r_t = \begin{cases} -2, & \text{if } \ell_t > 0 \\ r_{1,t} - (1 - r_{2,t})^2, & \text{otherwise} \end{cases} \tag{6}$$

3) Training: Our agent consists of an actor network determining a deterministic action $a = \mu(s;\theta^\mu)$ parameterised by θ^μ and a critic network to approximate the expected future return, defined as $Q(s,a;\theta^Q) = \mathbb{E}[R_t|s_t,a_t]$ with parameters θ^Q. The return R_t is the sum of discounted future return, i.e., $R_t = \sum_{i=t}^{T} \gamma^{i-t} r(s_i,a_i)$. Approaching to the approximation is done by updating the critic network through minimising the mean square error between its output and a target function (y_t), shown as follows:

$$L(\theta^Q) = \mathbb{E}[(Q(s_t,a_t;\theta^Q) - y_t)^2]$$

$$y_t = r(s_t,a_t) + \gamma Q^-(s_{t+1}, \mu^-(s_{t+1};\theta^{\mu^-});\theta^{Q^-})$$

where $Q^-(s,a;\theta^-)$ and $\mu^-(s;\theta^{\mu^-})$ are target critic network and target policy network. Their parameters are updated by calculating the exponetially weighted moving average (EWMA) of critic network and policy network, i.e., $\theta^- \leftarrow \tau \times \theta + (1-\tau) \times \theta^-$ where the τ is the weight parameter. Subsequently, with the updated critic network, policy performance of current actor network is predicted, expressed as $J(\theta^\mu) = \mathbb{E}[Q(s_t,\mu(s_t;\theta^\mu))]$. Training an actor network is done by updating its paramters through policy performance gradient ascent. Specifically, by applying the chain rule, the gradient of the policy performance, is computed as:

$$\nabla_{\theta^\mu} J(\theta^\mu) = \mathbb{E}[\nabla_{a_t} Q(s_t,\mu(s_t;\theta^\mu))\nabla_{\theta^\mu}\mu(s_t;\theta^\mu)]$$

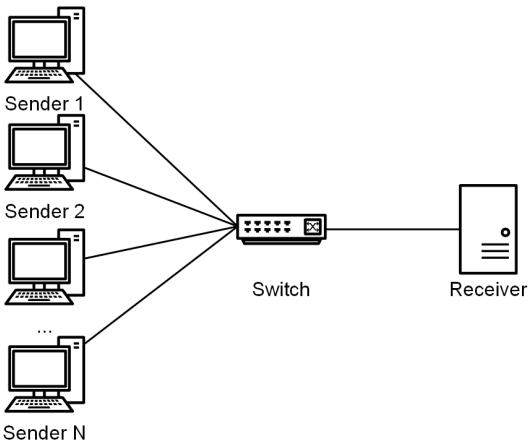


Fig. 3: A multi-to-one flow scenario.

Fig. 4: Cumulative reward per episode.

D. Window Enforcement Module

For an RL-controlled CC, it is important that waiting agent for action generation should not block the TCP normal behaviour. Therefore, we decouple the Window Decision Module, Receiver Measurement Module and standard feature to ensure none of them will block any other. Therefore, an ACK response will not be delayed in our scheme. In our design, an action request message will be sent to agent periodically by Window Decision Module. The resulting action is a factor within $[0,1]$. Once the Window Decision Module receive the action, it will extract the latest $cwnd$ in Receiver Measurement Module and compute the enforced congestion window ($cwnd_\mu$) using the following:

$$cwnd_\mu = cwnd \times scaling\_factor^a,\ a \in [0,1]$$

Then $cwnd_\mu$ will be included into the ACK header and take effect at the sender side.

IV. Simulation Setup and Results

Our scheme is implemented in NS-3 simulator. With the motivation to improve the performance of multiple flows to one receiver. We set up our topology as shown in Fig. 3. We compare our scheme with existing CC algorithms, i.e., NewReno, Cubic, and BBR.

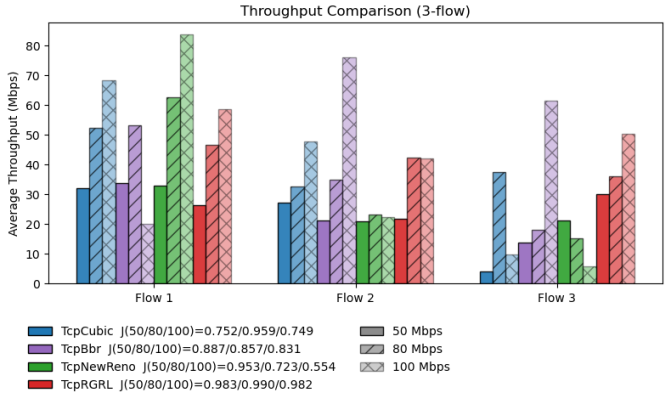


Fig. 5: Throughput comparison under different bottleneck bandwidths.

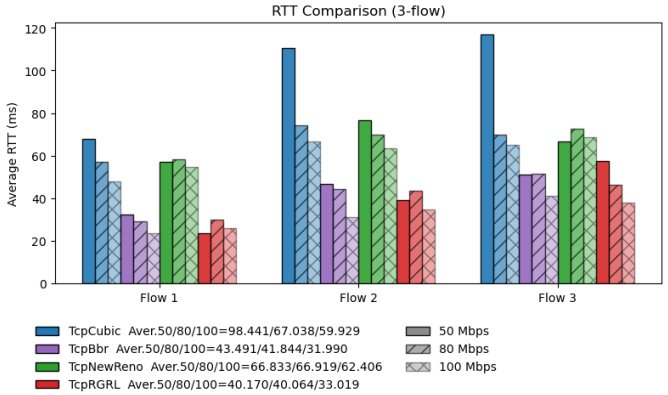


Fig. 6: RTT comparison under different bottleneck bandwidths.

A. 3-flow scenario

In this schenario, we started three flows with a 10s interval. The three flows are active respectively in $[0-50s]$, $[10-40s]$, and $[20-30s]$, in which case we focus on the fairness performance of our scheme. We set the bandwidth in our topology respectively as 50 Mbps, 80 Mbps, and 100 Mbps. The base RTT is set to 20 ms in all simulations. We trained our agent for 100 episodes where each is a simulation in NS-3. The reward per episode can be seen in Fig. 4. The average throughput and fairness performance of our scheme can be seen in Fig. 5. For the fairness we calculated Jain Index for the three flows in their active period. We can see that out scheme RGRL can have a better fairness than other CC algorithm in all three bandwidth settings, i.e., around 0.98. Meanwhile, it is observed that BBR can achieve 0.88 in 50 Mbps case, which is decreased slightly when the bandwidth is lower, particularly, to 0.83 for 100 Mbps setting. NewReno Jain index is decreased significantly when the bandiwidth is higher, from 0.88 in 50 Mbps to 0.5536 in 100 Mbps. Cubic achieves a good performance, 0.9592 only in 80 Mbps setting while remains lower than 0.80 in other cases. Overall, RGRL advantage is more significant when the bandwidth is higher.

We also present the average RTT of flow for each CC algorithm that can be seen in Fig. 6. Despite Cubic and NewReno achieves satisfying fairness performance in specific setting, they are out of the cost of RTT from recklessly increasing $cwnd$, i.e., over 60 ms for NewReno and at least 59.93 for Cubic among the cases. The latter can achieve even 98.44 ms in 50 Mbps setting. While our scheme can achieve a low average RTT better than BBR in 50 Mbps and 80 Mbps. For the 100 Mbps, our scheme can achieve preferable performance to 33.02 ms similar to BBR's 31.99 ms. In terms of convergence, BBR has problem that one flow can get lower share of the bandwidth that is not significantly shown in jain index but in real time throughput. The throughput over time in the three settings can be seen in Fig 7, Fig. 8, and Fig. 9.

B. FCT in random flow scenario

V. Conclusion

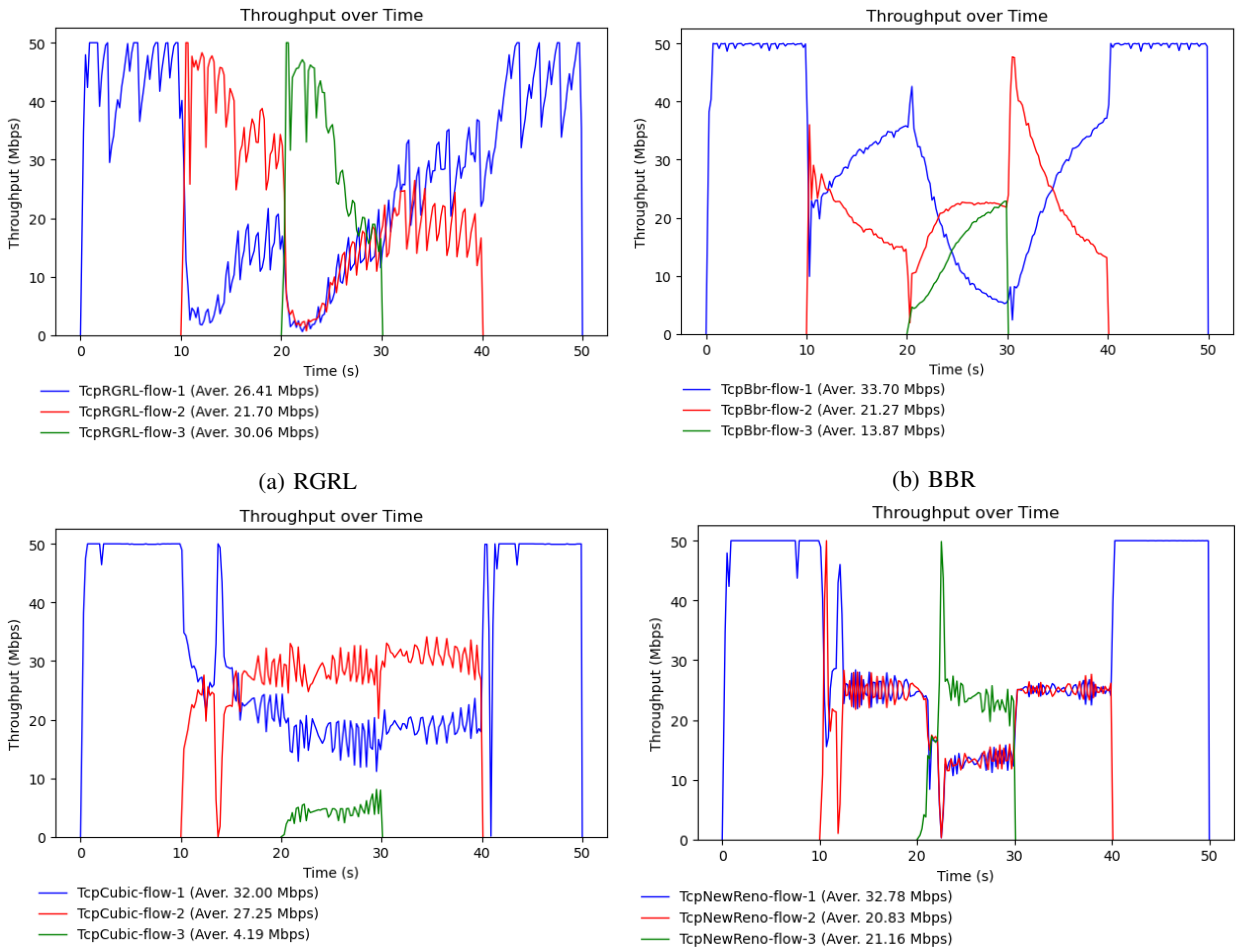
The conclusion goes here.

Acknowledgment

The authors would like to thank...

References

[1] H. Kopka and P. W. Daly, *A Guide to LaTeX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.



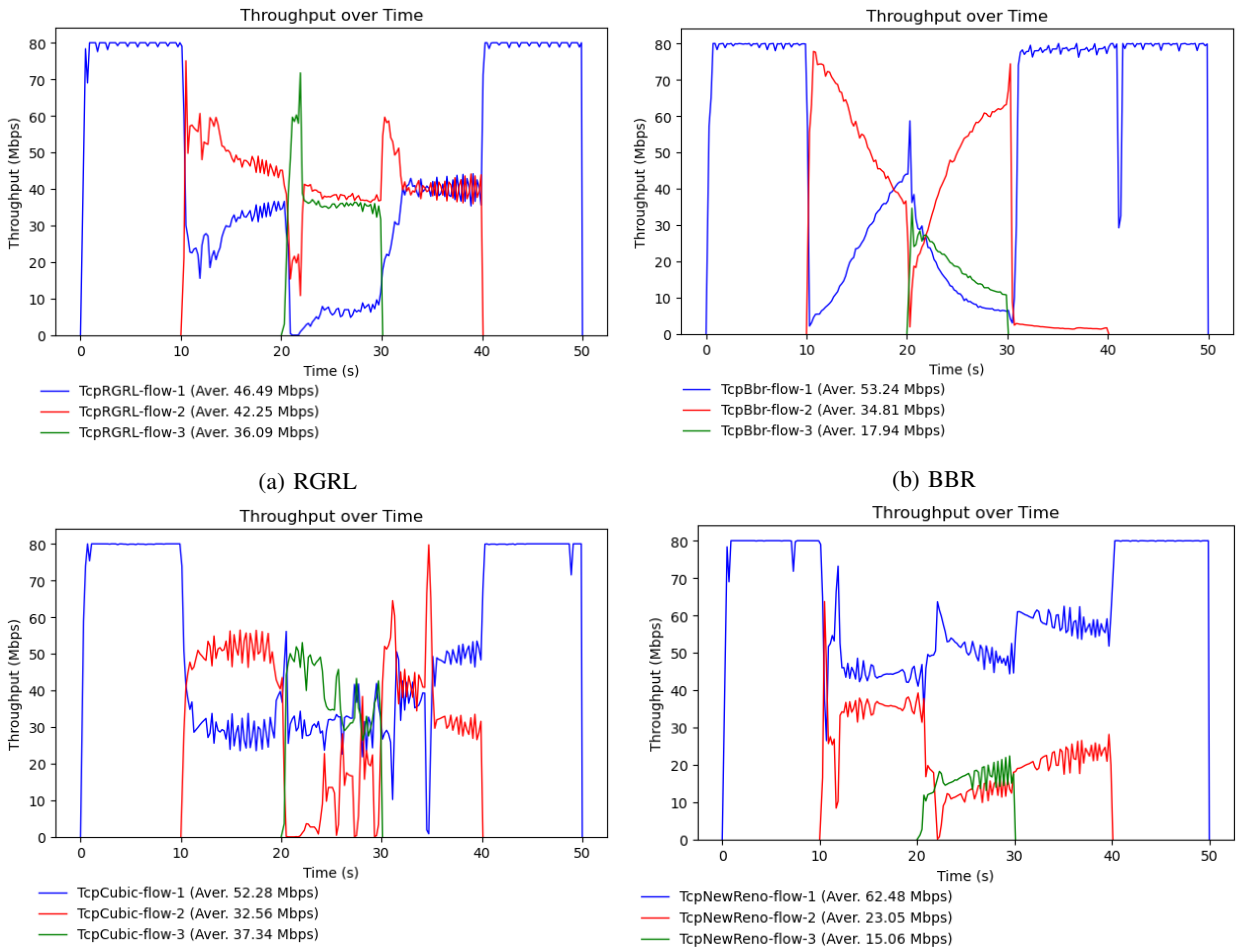
(a) RGRL

(b) BBR

(c) Cubic

(d) NewReno

Fig. 7: Throughput and fairness performance of our scheme and baselines in 3-flow scenario with bandwidth set to 50 Mbps.



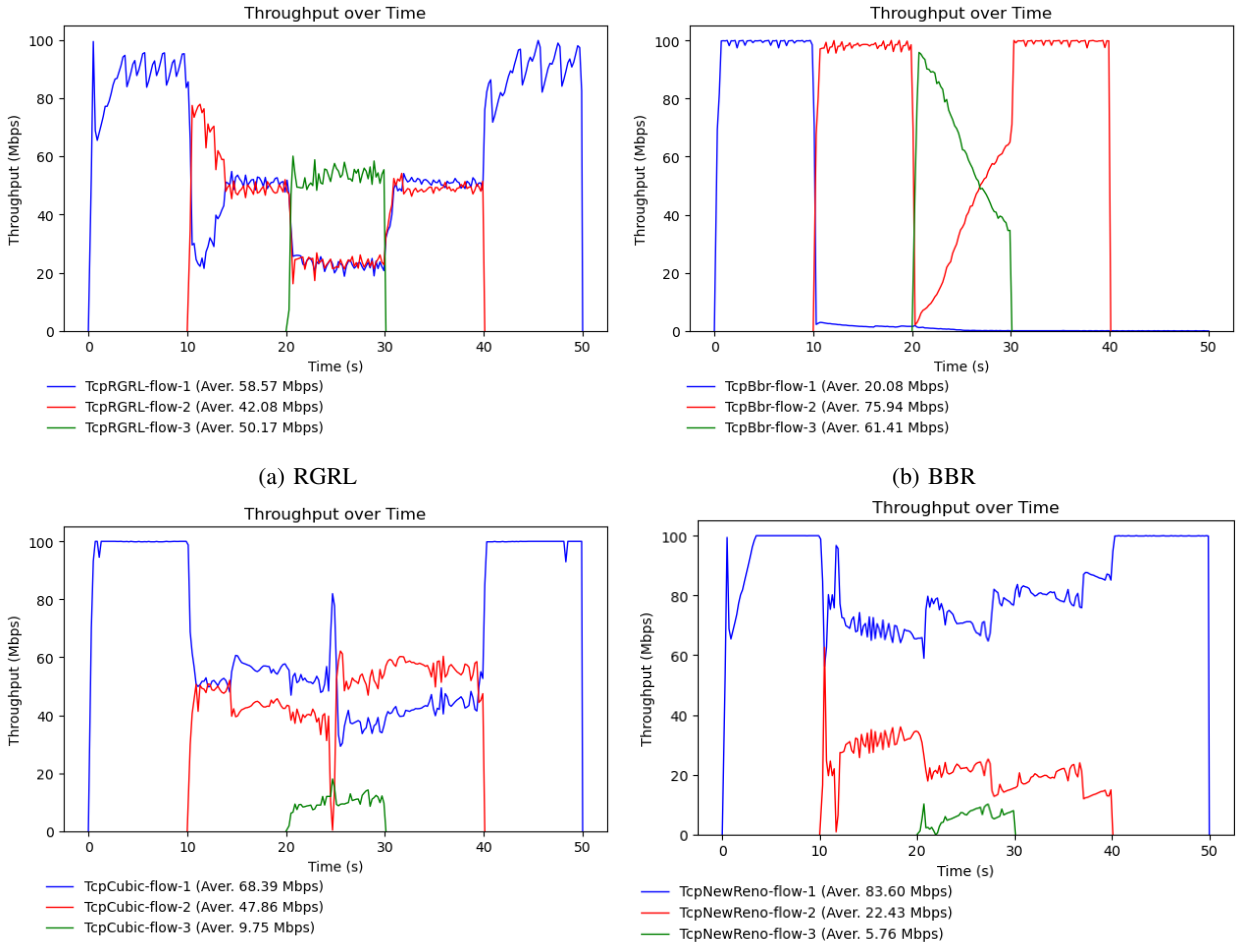
(a) RGRL

(b) BBR

(c) Cubic

(d) NewReno

Fig. 8: Throughput and fairness performance of our scheme and baselines in 3-flow scenario with bandwidth set to 80 Mbps.



(a) RGRL

(b) BBR

(c) Cubic

(d) NewReno

Fig. 9: Throughput and fairness performance of our scheme and baselines in 3-flow scenario with bandwidth set to 100 Mbps.